UNIT II

FUNCTIONS, CLASSES & OBJECTS IN C++

Complete Debugging Solutions, Corrected C++ Code,
Pseudo-Code, Dry Runs and Expected Outputs

Prepared from the supplied Unit II question bank

How to Use This Document

- Section A answers all five debugging questions with defect classification, C++ rule explanation, corrected code, and expected output where the question requests it.
- Section B answers all five pseudo-code design questions with class structure, explicit data types and parameter passing, and trace tables.
- Where the question bank does not provide sample values, the document states reasonable sample assumptions before tracing.
- Line numbers are described by the relevant statements because exact numbering can change with formatting.

Important correction: the Unit II Question A3 overload set is inherently ambiguous for a one-argument call because both the defaulted cuboid overload and the one-argument cube overload are viable. The corrected design below removes that conflict rather than relying on compiler-specific behavior.

SECTION A — DEBUGGING QUESTIONS

Question 1 — Dangling Reference in Return-by-Reference

1. Error analysis

- Line 8 (`return result;`) — Undefined behavior / dangling reference. `result` is an automatic local variable. It is destroyed when getMaximumScaled returns, so returning `int&` refers to an object whose lifetime has ended.
- Line 19 — The reference binds to the invalid returned reference. The declaration itself may compile, but using the reference after the function returns is undefined behavior.
- Line 27 — Runtime undefined behavior. Reading danglingRef may print an arbitrary value, appear to work, or crash. There is no guaranteed output for the buggy program.
- getArrayElement is safe when the index is valid because the array belongs to the caller and remains alive after the function returns. The function returns a reference to an existing array element, not a local copy. Production code should also validate the index.

2. Corrected complete C++ program

```
#include <iostream>
#include <stdexcept>

// Return by value: the computed result is a new value owned by the caller.
int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor;
}

// Safe reference return: refers to an element in the caller-owned array.
int& getArrayElement(int arr[], int size, int index) {
    if (index < 0 || index >= size) {
        throw std::out_of_range("Array index out of range");
    }
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    int scaledValue = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaledValue << std::endl;

    int data[3] = {100, 200, 300};
```

```
    getArrayElement(data, 3, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    std::cout << "Scaled Value After Array Call: "
              << scaledValue << std::endl;

    return 0;
}
```

3. Dry run and expected output

Step	Operation	Result
1	Compare x=10 and y=20; maximum is 20.	maxVal = 20
2	Multiply maximum by scaleFactor=2; return by value.	scaledValue = 40
3	Assign 999 through getArrayElement(data,3,1).	data = {100,999,300}
4	Read scaledValue again; it is an independent int.	40

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value After Array Call: 40
```

Question 2 — Static Data Member Linkage and Member Function Scoping

1. Error analysis

- Static data member declaration — Linker error in the traditional out-of-class definition model: `static double totalBankReserves;` inside the class declares the member, but does not provide its required storage definition. Define it once at namespace/file scope.
- Line 19 (`accHolder` inside displayAudit) — Compilation error: displayAudit is static and has no `this` pointer. It cannot access an individual object's non-static field without an object.
- Separate class-wide and object-specific behavior: displayAudit prints total reserves; a non-static displayAccount method prints one account holder and that account's balance.

2. Corrected complete C++ program

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(const std::string& name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder << std::endl;
        std::cout << "Account Balance: $" << balance << std::endl;
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << std::endl;
    }
```

```
};

// Required definition and initialization of the static data member.
double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();

    return 0;
}
```

3. Expected output

```
Account Holder: Alice
Account Balance: $5000
Account Holder: Bob
Account Balance: $3000
Total Reserves in Vault: $8000
```

Note: C++17 and later also allow an inline static data member definition inside the class (`inline static double totalBankReserves = 0.0;`). The code above uses the conventional external definition requested by the question.

Question 3 — Function Overload Ambiguity via Default Arguments

1. Error analysis

- Line 19 — Compilation error: `computeVolume(5.0)` matches both the cuboid overload `computeVolume(double, double = 1.0, double = 1.0)` and the cube overload `computeVolume(double)`. Both are viable with one argument, so overload resolution cannot choose uniquely.
- Line 22 — The two-argument call also conflicts with the cuboid overload and the cylinder overload: both accept two double arguments. Their parameter names and intended meanings do not participate in overload resolution.
- Correct design: use distinct function names (`computeCuboidVolume`, `computeCubeVolume`, `computeCylinderVolume`) so meaning is explicit and no default-argument overload ambiguity remains.

2. Corrected complete C++ program

```
#include <iostream>

double computeCuboidVolume(double length,
                           double width = 1.0,
                           double height = 1.0) {
    return length * width * height;
}

double computeCubeVolume(double side) {
    return side * side * side;
}

double computeCylinderVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

int main() {
    std::cout << "Cuboid: " << computeCuboidVolume(5.0) << std::endl;
    std::cout << "Cube: " << computeCubeVolume(5.0) << std::endl;
```

```
    std::cout << "Cylinder: "
              << computeCylinderVolume(3.0, 7.0) << std::endl;
    return 0;
}
```

3. Dry run and expected output

Call	Calculation	Result
computeCuboidVolume(5.0)	5 × default 1 × default 1	5
computeCubeVolume(5.0)	5 × 5 × 5	125
computeCylinderVolume(3.0,7.0)	3.14159 × 3 × 3 × 7	197.92017

```
Cuboid: 5
Cube: 125
Cylinder: 197.92
```

The cylinder value is printed using the default stream precision; the underlying result is approximately 197.92017.

Question 4 — Array of Objects Missing Default Constructor

1. Error analysis

- Line 20 (`Student batch[3];`) — Compilation error: this declaration requires three Student objects to be default-constructed. Because the class declares a parameterized constructor, the compiler does not implicitly generate a zero-argument default constructor.
- Technique A: explicitly add `Student()`, initializing members to safe default values. Then the array declaration is valid and each element can later be assigned a parameterized temporary.
- Technique B: initialize the array directly with parameterized constructor calls, e.g. `Student batch[3] = { Student(101, "Aarav"), ... };`. This does not require a default constructor.

2. Corrected program — Technique A

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("Not Assigned") {}

    Student(int r, const std::string& n)
        : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber
                  << ", Name: " << name << std::endl;
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
```

```
        batch[i].display();
    }

    return 0;
}
```

3. Dry run and expected output

Array index	Assignment	Stored object
0	Student(101, "Aarav")	101, Aarav
1	Student(102, "Bhavna")	102, Bhavna
2	Student(103, "Chirag")	103, Chirag

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5 — Friend Function Declaration and Scope Resolution Flaws

1. Error analysis

- Before class Engine — Compilation issue: `TelemetryAuditor` is used in the friend function parameter list before the compiler has seen its declaration. Add `class TelemetryAuditor;` as a forward declaration.
- Line 29 — Compilation error: `alertThreshold` is a private non-static member of TelemetryAuditor. A friend function has access privileges but must still name the object whose member it accesses: `a.alertThreshold`.
- Friend function mechanics — A friend is a non-member function, not a method. It has no implicit `this` pointer and is not called using `v8.inspectDiagnostics(...)`.
- Invocation — Call it as `inspectDiagnostics(v8, audit);`. Both classes must grant friendship to the same function declaration.

2. Corrected complete C++ program

```
#include <iostream>

class TelemetryAuditor; // Forward declaration

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    explicit TelemetryAuditor(int threshold)
        : alertThreshold(threshold) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};
```

```
// Non-member friend function: explicitly receives both objects.
void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at "
                  << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);
    return 0;
}
```

3. Expected output

```
Warning: Engine overheat!
```

SECTION B — PSEUDO-CODE WRITING QUESTIONS

The pseudo-code below follows the supplied question bank's class names and requirements. It distinguishes instance attributes from STATIC class attributes and indicates REF parameters or return values where relevant.

Question 1 — Corporate Vault Management with Static Data and Functions

1. Design overview

- Each BranchOffice object stores its own branch ID, name, and current balance.
- totalVaultReserves and totalBranchesActive are STATIC values shared by every branch object.
- Constructor and successful deposits/withdrawals update both the instance balance and shared reserve total.
- Static methods report enterprise-wide data and calculate the ratio against a supplied target.

2. Structured pseudo-code

```
CLASS BranchOffice
    PRIVATE:
        branchID : STRING
        branchName : STRING
        currentBalance : REAL

        STATIC totalVaultReserves : REAL <- 0.0
        STATIC totalBranchesActive : INTEGER <- 0

    PUBLIC:
        CONSTRUCTOR(id : STRING, name : STRING, openingDeposit : REAL)
            branchID <- id
            branchName <- name
            currentBalance <- openingDeposit
            totalVaultReserves <- totalVaultReserves + openingDeposit
            totalBranchesActive <- totalBranchesActive + 1
        END CONSTRUCTOR

        METHOD Deposit(amount : REAL)
            IF amount > 0 THEN
                currentBalance <- currentBalance + amount
                totalVaultReserves <- totalVaultReserves + amount
                DISPLAY "Deposit successful"
            ELSE
                DISPLAY "Invalid deposit amount"
            ENDIF
        END METHOD

        METHOD Withdraw(amount : REAL) RETURNS BOOLEAN
            IF amount > 0 AND amount <= currentBalance THEN
                currentBalance <- currentBalance - amount
                totalVaultReserves <- totalVaultReserves - amount
                RETURN TRUE
            ELSE
                RETURN FALSE
            ENDIF
        END METHOD

        STATIC METHOD GetVaultSummary()
            DISPLAY "Active Branches: ", totalBranchesActive
            DISPLAY "Total Vault Reserves: ", totalVaultReserves
        END METHOD

        STATIC METHOD GetGlobalLiquidityRatio(targetReserve : REAL)
            IF targetReserve <= 0 THEN
                RETURN 0.0
            ENDIF
```

```
            RETURN totalVaultReserves / targetReserve
        END STATIC METHOD
ENDCLASS

BEGIN
    DECLARE branch1 : BranchOffice
    DECLARE branch2 : BranchOffice
    DECLARE withdrawalStatus : BOOLEAN
    DECLARE liquidity : REAL

    branch1 <- NEW BranchOffice("B001", "Central", 500000.0)
    branch2 <- NEW BranchOffice("B002", "North", 300000.0)

    branch1.Deposit(150000.0)
    withdrawalStatus <- branch2.Withdraw(50000.0)

    BranchOffice.GetVaultSummary()
    liquidity <- BranchOffice.GetGlobalLiquidityRatio(1000000.0)
    DISPLAY "Global Liquidity Ratio: ", liquidity
END
```

3. Trace table

Sample assumption: Branch 1 opens with $500,000 and Branch 2 opens with $300,000 because opening deposits were not specified in the question.

Event	Branch 1 balance	Branch 2 balance	Static total reserves	Active branches
Initial construction	500,000	300,000	800,000	2
Branch 1 deposits 150,000	650,000	300,000	950,000	2
Branch 2 withdraws 50,000	650,000	250,000	900,000	2
Liquidity ratio vs. 1,000,000	—	—	900,000 / 1,000,000 = 0.90	2

Final consolidated reserves = $900,000; global liquidity ratio = 0.90 (90%).

Question 2 — Matrix Mathematical Engine and Friend Dot Product

1. Design overview

- Matrix2D stores a private 3×3 REAL array.
- Initialize, SetElement, GetElement, and Display are declared in the class and implemented outside it using Matrix2D::MethodName.
- ComputeCrossTrace is a non-member friend function. It receives two matrices by CONST REF and directly accesses their private diagonal elements.

2. Structured pseudo-code

```
CLASS Matrix2D
    PRIVATE:
        grid : ARRAY[3][3] OF REAL

    PUBLIC:
        METHOD Initialize()
        METHOD SetElement(row : INTEGER, col : INTEGER, value : REAL)
        METHOD GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
        METHOD Display()

        FRIEND FUNCTION ComputeCrossTrace(
            A : CONST REF Matrix2D,
            B : CONST REF Matrix2D
        ) RETURNS REAL
ENDCLASS
```

```
METHOD Matrix2D::Initialize()
    FOR row <- 0 TO 2 STEP 1
        FOR col <- 0 TO 2 STEP 1
            grid[row][col] <- 0.0
        ENDFOR
    ENDFOR
END METHOD

METHOD Matrix2D::SetElement(row : INTEGER, col : INTEGER, value : REAL)
    IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
        grid[row][col] <- value
    ELSE
        DISPLAY "ERROR: Matrix index out of bounds"
    ENDIF
END METHOD

METHOD Matrix2D::GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
    IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
        RETURN grid[row][col]
    ELSE
        DISPLAY "ERROR: Matrix index out of bounds"
        RETURN 0.0
    ENDIF
END METHOD

METHOD Matrix2D::Display()
    FOR row <- 0 TO 2 STEP 1
        FOR col <- 0 TO 2 STEP 1
            DISPLAY grid[row][col], " "
        ENDFOR
        DISPLAY NEWLINE
    ENDFOR
END METHOD

FUNCTION ComputeCrossTrace(A : CONST REF Matrix2D,
                           B : CONST REF Matrix2D) RETURNS REAL
    DECLARE total : REAL <- 0.0
    FOR i <- 0 TO 2 STEP 1
        total <- total + (A.grid[i][i] * B.grid[i][i])
    ENDFOR
    RETURN total
END FUNCTION

BEGIN
    DECLARE A : Matrix2D
    DECLARE B : Matrix2D
    A.Initialize()
    B.Initialize()

    // Set diagonal values for the sample.
    A.SetElement(0,0,2); A.SetElement(1,1,4); A.SetElement(2,2,6)
    B.SetElement(0,0,1); B.SetElement(1,1,3); B.SetElement(2,2,5)

    DISPLAY "Cross Trace: ", ComputeCrossTrace(A, B)
END
```

3. Friend function trace

i	A[i][i]	B[i][i]	Product	Running total
0	2	1	2 × 1 = 2	2
1	4	3	4 × 3 = 12	14
2	6	5	6 × 5 = 30	44

ComputeCrossTrace(A,B) returns 44.

Question 3 — Student Academic Roster Using an Array of Objects

1. Design overview

- Create three StudentRecord objects using a default constructor.
- ReadData fills the roll number, name, and three marks, then calls the private CalculateResult method.
- Percentage is the arithmetic average of the three marks. Grade thresholds are applied in descending order.
- Bubble Sort compares GetPercentage values and swaps whole objects so the roster is sorted from highest to lowest percentage.

2. Structured pseudo-code

```
CLASS StudentRecord
    PRIVATE:
        rollNo : INTEGER
        studentName : STRING
        marks : ARRAY[3] OF REAL
        percentage : REAL
        grade : CHARACTER

        METHOD CalculateResult()
            percentage <- (marks[0] + marks[1] + marks[2]) / 3.0

            IF percentage >= 85 THEN
                grade <- 'A'
            ELSE IF percentage >= 70 THEN
                grade <- 'B'
            ELSE IF percentage >= 50 THEN
                grade <- 'C'
            ELSE
                grade <- 'F'
            ENDIF
        END METHOD

    PUBLIC:
        CONSTRUCTOR()
            rollNo <- 0
            studentName <- ""
            marks[0] <- 0
            marks[1] <- 0
            marks[2] <- 0
            percentage <- 0.0
            grade <- 'F'
        END CONSTRUCTOR

        METHOD ReadData(r : INTEGER, name : STRING,
                        m1 : REAL, m2 : REAL, m3 : REAL)
            rollNo <- r
            studentName <- name
            marks[0] <- m1
            marks[1] <- m2
            marks[2] <- m3
            CALL CalculateResult()
        END METHOD

        METHOD GetPercentage() RETURNS REAL
            RETURN percentage
        END METHOD

        METHOD Display()
            DISPLAY rollNo, " | ", studentName, " | ",
```

```
                    percentage, "% | Grade ", grade
        END METHOD
ENDCLASS

BEGIN
    DECLARE students : ARRAY[3] OF StudentRecord
    DECLARE i, pass : INTEGER
    DECLARE temp : StudentRecord

    students[0].ReadData(101, "Asha", 90, 88, 92)
    students[1].ReadData(102, "Bala", 75, 80, 70)
    students[2].ReadData(103, "Charan", 60, 65, 55)

    FOR pass <- 0 TO 1 STEP 1
        FOR i <- 0 TO 1 - pass STEP 1
            IF students[i].GetPercentage()
               < students[i+1].GetPercentage() THEN
                temp <- students[i]
                students[i] <- students[i+1]
                students[i+1] <- temp
            ENDIF
        ENDFOR
    ENDFOR

    DISPLAY "RANK | ROLL | NAME | PERCENTAGE | GRADE"
    FOR i <- 0 TO 2 STEP 1
        DISPLAY i + 1, " | "
        students[i].Display()
    ENDFOR
END
```

3. Sample calculations and sorting trace

Student	Marks	Percentage	Grade
Asha	90, 88, 92	90.00%	A
Bala	75, 80, 70	75.00%	B
Charan	60, 65, 55	60.00%	C

Pass	Comparison	Swap?	Order after pass
Initial	—	—	Asha (90), Bala (75), Charan (60)
1	90 < 75? No	No	Asha, Bala, Charan
1	75 < 60? No	No	Asha, Bala, Charan
2	90 < 75? No	No	Asha, Bala, Charan

Final rank: 1. Asha — 90%; 2. Bala — 75%; 3. Charan — 60%. No swaps are needed for this already descending sample. The Bubble Sort logic still checks adjacent pairs on each pass.

Question 4 — Flexible Logger with Overloaded Functions and Inline Helpers

1. Design overview

- FormatPrefix is an INLINE helper that formats a tag as [TAG] .
- To avoid ambiguity between a one-argument overload and a second overload with default arguments, use one message overload with default severity and timestamp. It handles one-, two-, or three-argument message calls.
- A separate two-argument hardware diagnostic overload accepts INTEGER errorCode followed by STRING systemModule. The argument types/order distinguish it from the message overload.

2. Structured pseudo-code

```
INLINE FUNCTION FormatPrefix(prefixTag : STRING) RETURNS STRING
```

```
    RETURN "[" + prefixTag + "] "
END FUNCTION

// Message overload: defaults allow calls with 1, 2, or 3 arguments.
FUNCTION LogEvent(message : STRING,
                  severityLevel : INTEGER = 1,
                  timestamp : STRING = "00:00:00")
    DECLARE severityName : STRING

    SWITCH severityLevel
        CASE 1:
            severityName <- "LOW"
            BREAK
        CASE 2:
            severityName <- "MEDIUM"
            BREAK
        CASE 3:
            severityName <- "CRITICAL"
            BREAK
        DEFAULT:
            severityName <- "UNKNOWN"
    ENDSWITCH

    DISPLAY FormatPrefix(severityName), "@ ", timestamp,
            " : ", message
END FUNCTION

// Hardware fault overload.
FUNCTION LogEvent(errorCode : INTEGER, systemModule : STRING)
    DISPLAY "[HARDWARE_FAULT] Module: ", systemModule,
            " -> Error Code: ", HEX(errorCode)
END FUNCTION

BEGIN
    LogEvent("System started")
    LogEvent("Temperature rising", 2)
    LogEvent("Motor shutdown", 3, "14:35:20")
    LogEvent(255, "MOTOR_DRIVER")
END
```

3. Call-resolution trace

Call	Selected function	Defaults used / output meaning
LogEvent("System started")	Message overload	severityLevel=1, timestamp=00:00:00; LOW
LogEvent("Temperature rising", 2)	Message overload	timestamp=00:00:00; MEDIUM
LogEvent("Motor shutdown", 3, "14:35:20")	Message overload	No defaults; CRITICAL at 14:35:20
LogEvent(255, "MOTOR_DRIVER")	Hardware overload	Error code formatted in hexadecimal (0xFF)

Representative output:

```
[LOW] @ 00:00:00 : System started
[MEDIUM] @ 00:00:00 : Temperature rising
[CRITICAL] @ 14:35:20 : Motor shutdown
[HARDWARE_FAULT] Module: MOTOR_DRIVER -> Error Code: 0xFF
```

Design note: the question bank's literal Overload 1 `LogEvent(message)` and Overload 2 `LogEvent(message, severity=1, timestamp=...)` cannot both coexist without ambiguity for a one-argument call. The corrected design merges the general-message behavior into the defaulted overload.

Question 5 — Hardware Register Bank with Return-by-Reference

1. Design overview

- RegisterBank owns eight integer register elements and an errorSink member.
- The constructor initializes every register to zero and errorSink to -1.
- GetRegister returns REF INTEGER. For a valid index, it returns the actual array element; for an invalid index, it returns the object's errorSink member.
- Because the returned reference designates a persistent member of the RegisterBank object, the client can assign through the function call. The RegisterBank object must remain alive while the reference is used.

2. Structured pseudo-code

```
CLASS RegisterBank
    PRIVATE:
        registers : ARRAY[8] OF INTEGER
        errorSink : INTEGER

    PUBLIC:
        CONSTRUCTOR()
            FOR i <- 0 TO 7 STEP 1
                registers[i] <- 0
            ENDFOR
            errorSink <- -1
        END CONSTRUCTOR

        METHOD GetRegister(index : INTEGER) RETURNS REF INTEGER
            IF index >= 0 AND index < 8 THEN
                RETURN REF registers[index]
            ELSE
                DISPLAY "ALERT: Register index out of range"
                RETURN REF errorSink
            ENDIF
        END METHOD

        METHOD DisplayState()
            DISPLAY "Register Bank (HEX):"
            FOR i <- 0 TO 7 STEP 1
                DISPLAY "R", i, " = ", HEX(registers[i])
            ENDFOR
            DISPLAY "Error Sink = ", errorSink
        END METHOD
ENDCLASS

BEGIN
    DECLARE bank : RegisterBank
    DECLARE readVal : INTEGER

    bank.GetRegister(0) <- 170
    bank.GetRegister(1) <- 255

    readVal <- bank.GetRegister(0)
    DISPLAY "Read Register 0: ", HEX(readVal)

    bank.GetRegister(10) <- 999
    DISPLAY "After invalid write, error sink: ", bank.GetRegister(10)

    bank.DisplayState()
END
```

3. Execution trace

Step	Operation	Register / errorSink state
Initial	Constructor	R0–R7 = 0; errorSink = -1
1	bank.GetRegister(0) <- 170	R0 = 170 = 0xAA
2	bank.GetRegister(1) <- 255	R1 = 255 = 0xFF
3	readVal <- bank.GetRegister(0)	readVal = 170 = 0xAA
4	bank.GetRegister(10) <- 999	Index invalid; returns reference to errorSink; errorSink becomes 999
5	DisplayState()	R0=0xAA, R1=0xFF, R2–R7=0x0; errorSink=999

Return-by-reference permits `functionCall() = value` because the function returns an lvalue reference to an existing assignable object. The assignment writes directly to that object. Returning a reference to a local automatic variable would be unsafe because that variable's lifetime ends when the function returns.

Illustrative display (hexadecimal formatting may include implementation-specific letter casing/prefix formatting):

```
Read Register 0: AA
ALERT: Register index out of range
After invalid write, error sink: 999
Register Bank (HEX):
R0 = AA
R1 = FF
R2 = 0
R3 = 0
R4 = 0
R5 = 0
R6 = 0
R7 = 0
Error Sink = 999
```

Quick Revision — Unit II Rules

Topic	Remember
Return by reference	Return references only to objects that outlive the call (caller-owned objects, static objects, or suitable persistent storage). Never return a reference to a local automatic variable.
Static data members	A traditional static data member needs one out-of-class definition; C++17 inline static members are an alternative.
Static member functions	No `this` pointer; access static members directly and instance members through an explicit object.
Default arguments and overloads	Defaults do not create a new overload signature. Avoid overload sets where multiple functions are viable for the same argument count/types.
Arrays of objects	An array declaration without element initializers requires an accessible default constructor for each element.
Friend functions	A friend is a non-member; it has no `this` pointer. It must use explicit object parameters to access private fields.
Scope resolution	Define outside-class members as `ReturnType ClassName::method(...)`.
Arrays inside classes	Validate indexes before accessing fixed-size arrays.
Bubble Sort objects	Swap whole objects based on a comparison key such as percentage.
Reference-returning accessors	The referenced object must remain alive; a member error sink is safe only while its owning object remains alive.

End of Unit II answer document.